Université de Carthage | Faculté des Sciences de Bizerte

Framework BigDATA

Tarek Ben Mena

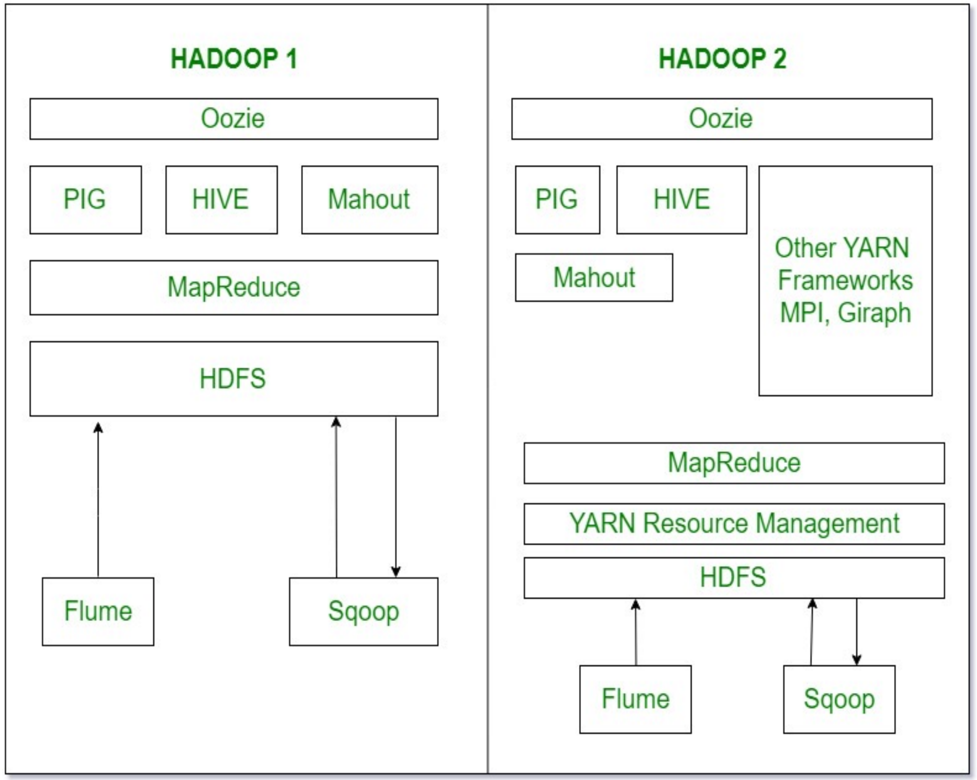
CONTEXTE

Contexte – le BigData

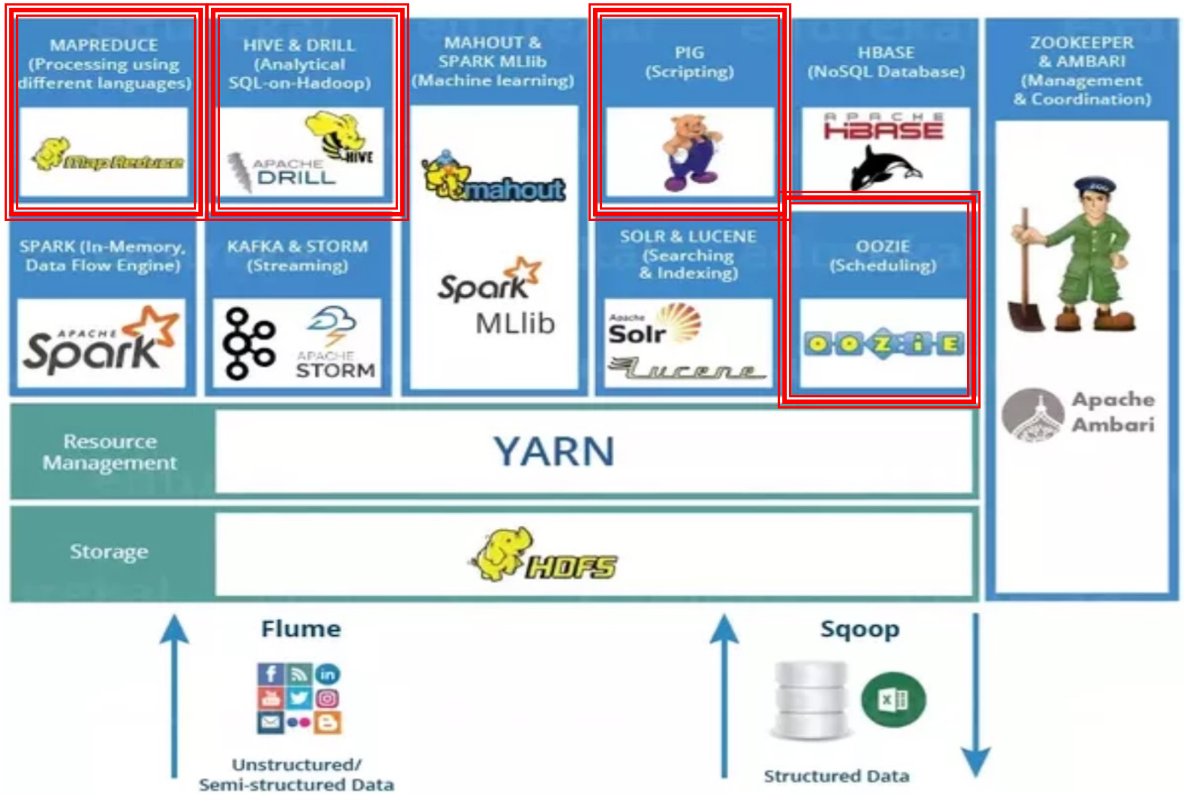
- Dans un contexte technologique ou les données sont très faciles à produire, l'analyse devient de plus en plus complexe.
- Les 1ère solutions à grosse volumétrie sont commerciales et se basent sur un modèle MPP (Massively Parallel Processing) ou shared nothing.
  - Exemple : Teradata (1983) , greenplum (2003) …
- Solutions limitées aux grandes compagnies en raison du coût
- En 2003/2004, Google publie deux articles
  - *MapReduce: Simplified Data Processing on Large Clusters*
  - The Google File System
- Naissance de Hadoop en 2009 (Doug Cutting / Mike Cafarella)

Context - Hadoop

- Hadoop est conçu sur plusieurs idées :
  - Développé en JAVA pour la portabilité
  - Traitement des données basé sur le paradigme Map/Reduce
  - Le même matériel pour le traitement et le stockage des donnée
  - Les pannes matérielles sont gérées au niveau logiciel
  - Exécuter le code applicatif au plus prêt des données
  - Séparer la logique du traitement des données (Slave) de la logique de distribution du calcul (Master)



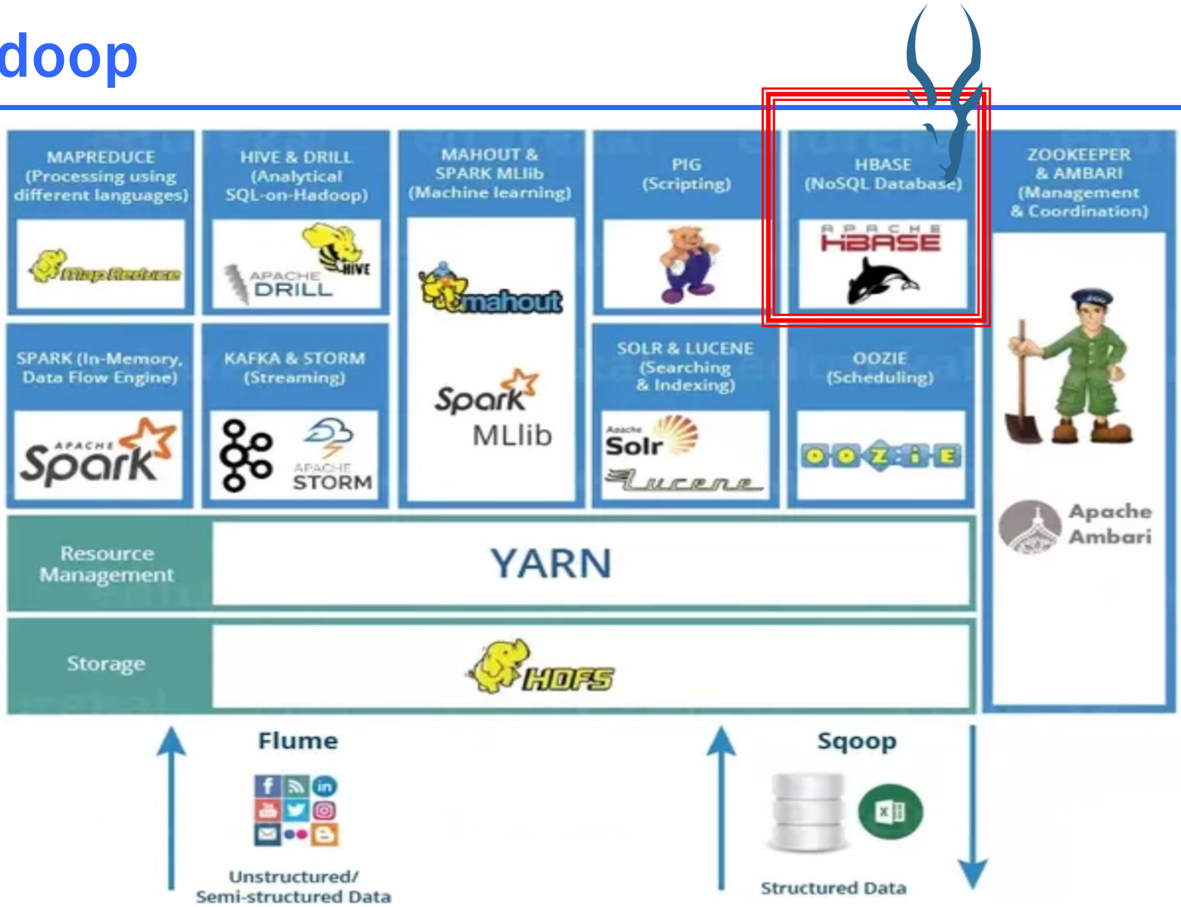
Context - Hadoop



Pig: Langage de script (Pig Latin)
Hive: Langage proche de SQL (Hive QL)
R Connectors: Accès à HDFS, exécution de requêtes Map/Reduce à partir du langage R

Mahout: bibliothèque de machine learning
Oozie: ordonnancer les jobs Map Reduce avec des workflows

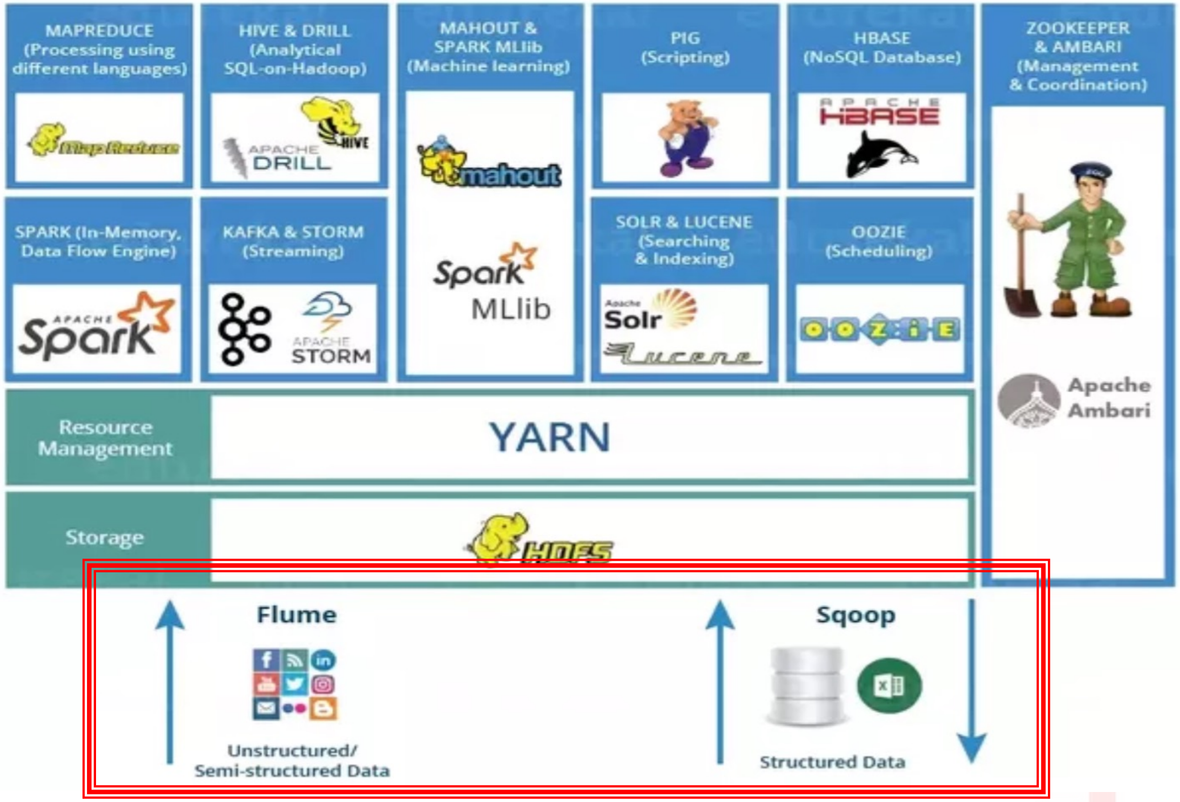
Context - Hadoop



Hbase : Base de données NoSQL orientée colonnes (table de +ieurs Milliard d’enregistrements avec des millions des colonnes

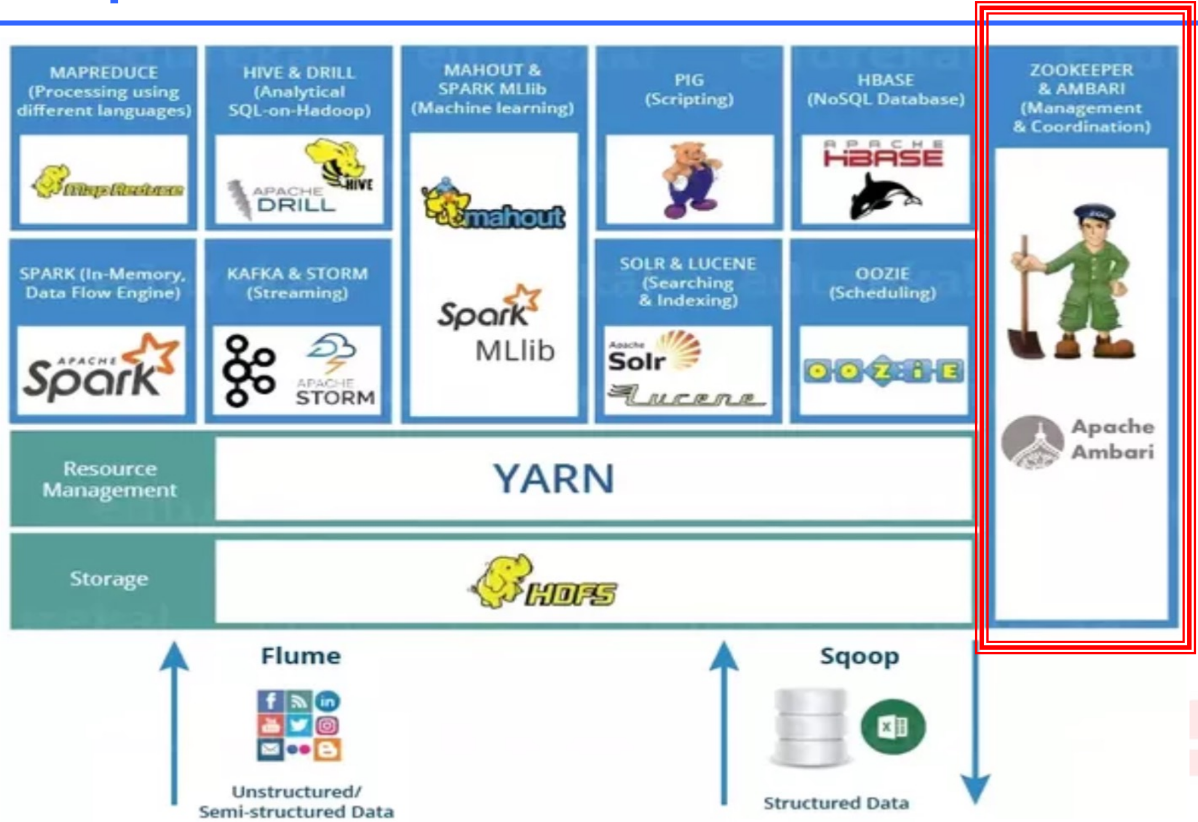
Impala : permet le requêtage de données directement à partir de HDFS (ou de Hbase) en utilisant des requêtes Hive SQL

Context - Hadoop



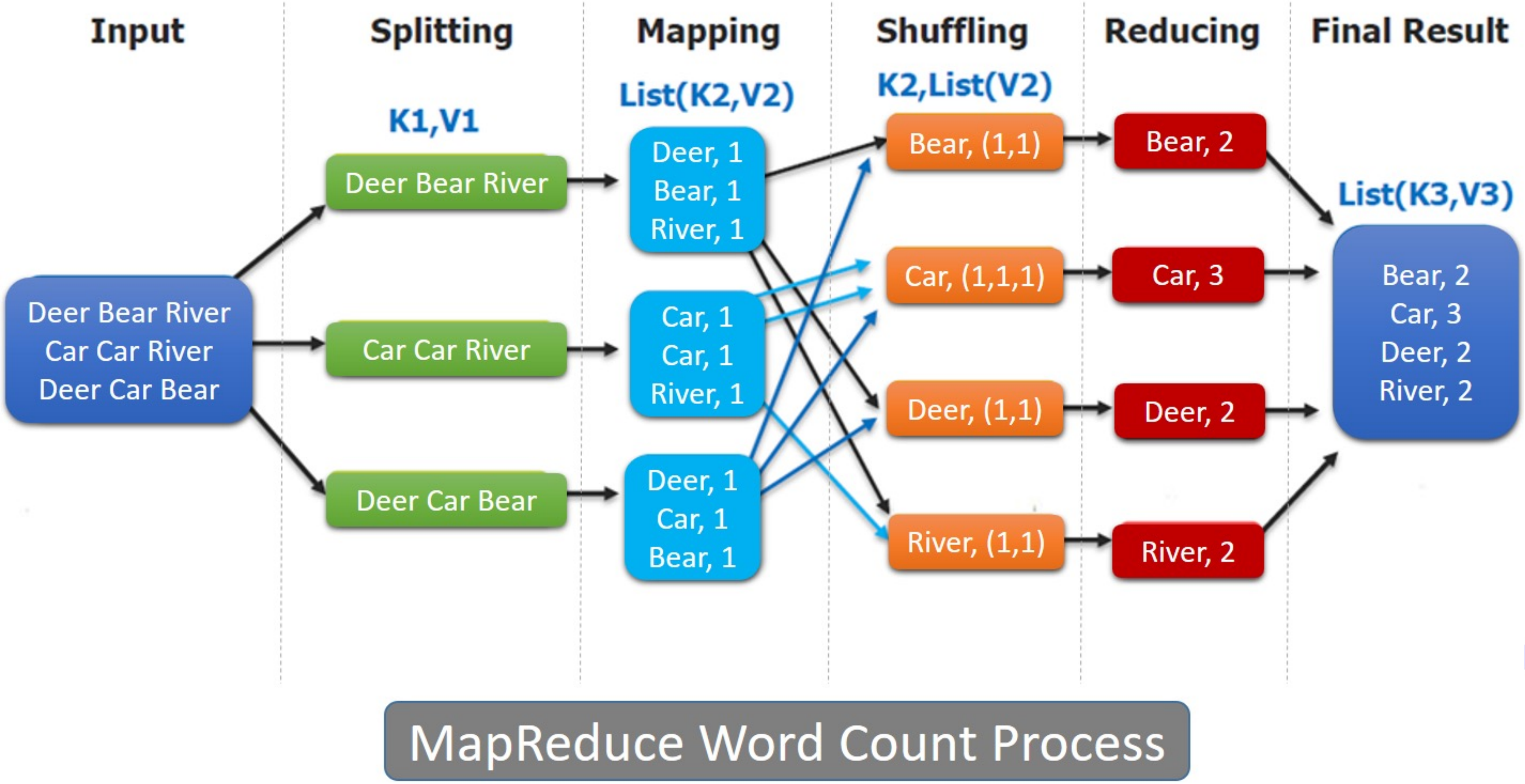
Sqoop: Lecture et écriture des données à partir de bases de données externes
Flume: Collecte de logs et stockage dans HDFS

Context - Hadoop



Ambari: outil pour le provisionnement, gestion et monitoring des clusters
Zookeeper: (Gestionnaire de Configuration) fournit un service centralisé pour maintenir les informations de configuration, de nommage et de synchronisation distribuée

Rappel - MapReduce



Rappel – MapReduce - Exemple

WordCount.java

```java
package com.exemple;

import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.fs.Path;

public class WordCount{
    public static class Map extends Mapper<LongWritable,Text,Text,IntWritab
        public void map(LongWritable key, Text value,Context context) throw
            String line = value.toString();
            StringTokenizer tokenizer = new StringTokenizer(line);
            while (tokenizer.hasMoreTokens()) {
                 value.set(tokenizer.nextToken());
                 context.write(value, new IntWritable(1));
            }
        }
    }
    public static class Reduce extends Reducer<Text,IntWritable,Text,IntWri
        public void reduce(Text key, Iterable<IntWritable> values,Context c
           int sum=0;
           for(IntWritable x: values)
           {
               sum+=x.get();
           }
           context.write(key, new IntWritable(sum));
        }
    }
```

```java
    public static void main(String[] args) throws Exception {
        Configuration conf= new Configuration();
        Job job = new Job(conf,"My Word Count Program");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(Map.class);
        job.setReducerClass(Reduce.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        Path outputPath = new Path(args[1]);
        //Configuring the input/output path from the filesystem into the jo
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        //deleting the output path automatically from hdfs so that we don't
        outputPath.getFileSystem(conf).delete(outputPath);
        //exiting the job only if the flag value becomes false
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

MapReduce limites

- La spécification de l'itération reste à la charge du programmeur;
  - Il faut stocker le résultat d'un 1er job dans une collection intermédiaire
  - puis réitérer le job en prenant la collection intermédiaire comme source.
- C'est laborieux pour l'implantation, et surtout très peu efficace quand la collection intermédiaire est grande.
- Le processus de sérialisation/dé-sérialisation sur disque propre à la gestion de la reprise sur panne en MapReduce entraîne des performances médiocres.

- Dans **Spark**, la méthode consiste à placer ces jeux de données en mémoire RAM et à éviter la pénalité des écritures sur le disque
  - Le défi est alors bien sûr de proposer une reprise sur panne automatique efficace

Traitement des données

Source de Données
- Document
- RDBMS
- Système de fichiers
- BD NoSql

Couche Intégration
- Connexion
- Collecte
- Correction
- Transformation

Couche Analyse
- Exploration
- Machine Learning
- Graph
- Streaming

Visualisation
- Report
- Graphique
- Times Series

C’EST QUOI SPARK?

Spark ? (1/2)

- C'est un moteur de traitement parallèle de données open source permettant d'effectuer des analyses rapides (In-memory) dédié au Big Data de manière distribuée (cluster computing).

- SPARK s'inspire fortement de Hadoop
  - L'utilisation de matériel commun pour le traitement mais ne gère pas le stockage des données
  - Les pannes matérielles sont gérées au niveau logiciel (RDD)
  - Exécuter le code applicatif au plus prêt des données
  - Séparer la logique du traitement des données de la logique de distribution du calcul
  - Intègre son propre cluster manager **SPARK standalone** lequel est interchangeable avec YARN / MESOS

Spark ? (2/2)

- Framework est en passe de remplacer Hadoop.
  - 100 x plus rapidement que Hadoop MapReduce in-memory,
  - 10 x plus vite sur disque
- Spark permet de traiter des données issues de
  - Hadoop Distributed File System (HDFS),
  - les bases de données NoSQL,
  - ou les data stores de données relationnels comme Apache Hive.
- Développé en Scala, langage basé sur une JVM.
  - 100 lignes de code JAVA peut être traduit en une dizaine de lignes en SCALA (d'ou la version Java 8)
  - Propose une API Java et Python
- Hérite des avantages de la programmation fonctionnelle
  - Le calcul est évalué comme une fonction mathématique
  - Le développement d'applications parallélisées est plus facile

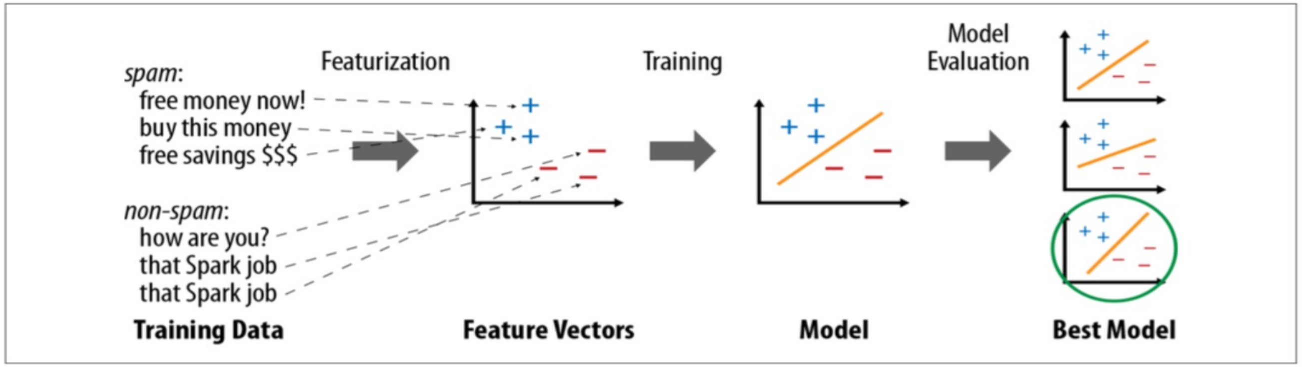
Spark : Spécification

- Un moteur d'exécution basé sur des opérateurs de haut niveau, comme Pig. Comprend
  - des opérateurs Map/Reduce, et
  - d'autres opérateurs de second ordre
- Introduit un concept de **collection résidente** en mémoire (RDD) qui améliore considérablement certains traitements, dont ceux basés sur des itérations (PageRank, kMeans)

- Fonctionnalités (Nos chapitre)
  - Batch / MapReduce
  - Spark SQL : exécution des requêtes en langages SQL
  - Streaming : le traitement de flux
  - MLlib Machine Learning : la fouille de données
  - GraphX : traitement des graphes

Spark : Fonctionnalités

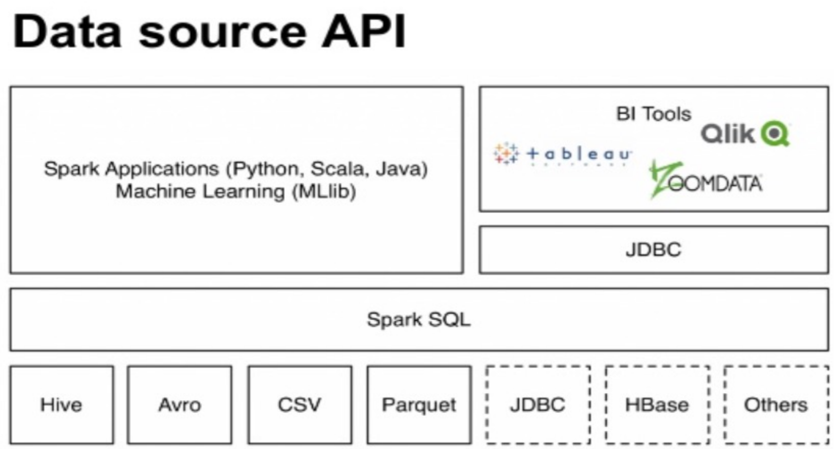
- MLlib



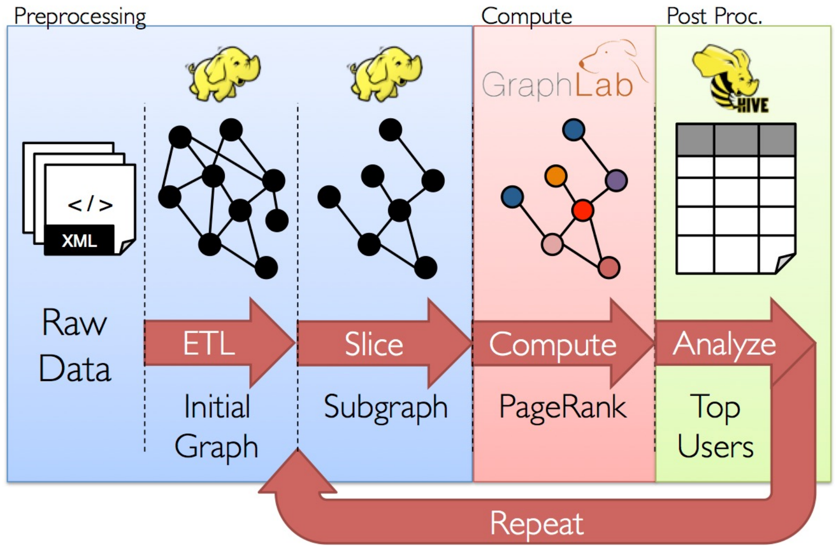
- Spark Stream

Spark : Fonctionnalités

- Spark SQL



- GraphX



Spark : Historique

- 2009 Initialement développé par l'AMPlab de l'université de Californie à Berkeley par Matei Zaharia
- Transmis à la fondation Apache, développement open-source depuis 2013
- En 2014, Spark a gagné le Daytona GraySort Contest7 dont l'objectif est de trier 100 To de données le plus rapidement possible. Ce record était préalablement détenu par Hadoop.
  - Spark (206 Machine, 23 min) vs Hadoop (2100 machines, 72 min)
- 2015 plus de 1000 collaborateurs, dont Intel, Facebook, IBM, Netflix
- Version actuelle : 3.5 . La version 2.0 a apporté des changements notables concernant les structures de données utilisées.
- Spark est utilisable avec plusieurs langages de programmation : Scala (natif), Java, Python, R, SQL.

INSTALLATION

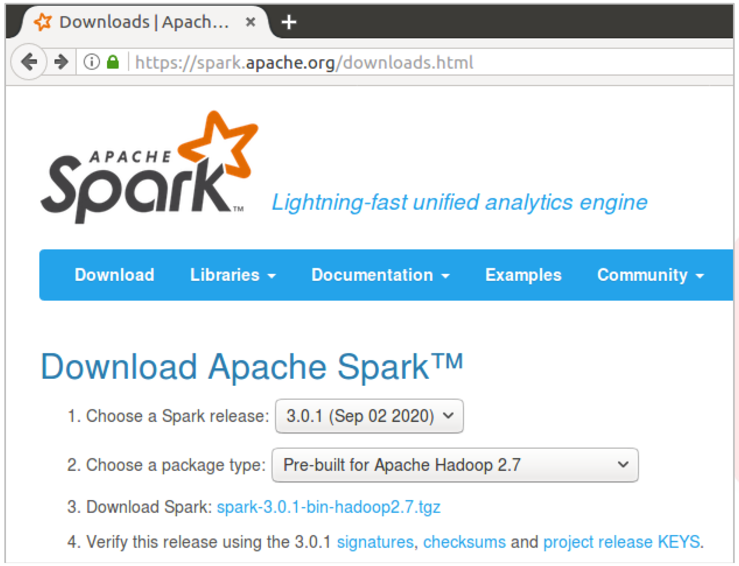
Installation Spark 3.0.1 sur Ubuntu 16.04

- Java est la seule dépendance pour installer Apache Spark

```
~$ sudo apt-get install default-jdk
```

```
spark@spark-VirtualBox:~$ java -version
openjdk version "1.8.0_265"
OpenJDK Runtime Environment (build 1.8.0_265-8u265-b01-0ubuntu2~16.04-b01)
OpenJDK Server VM (build 25.265-b01, mixed mode)
```

- Télécharger Spark
  - https://www.apache.org/dyn/closer.lua/spark/spark-3.0.1/spark-3.0.1-bin-hadoop2.7.tgz



Installation Spark 3.0.1 sur Ubuntu 16.04

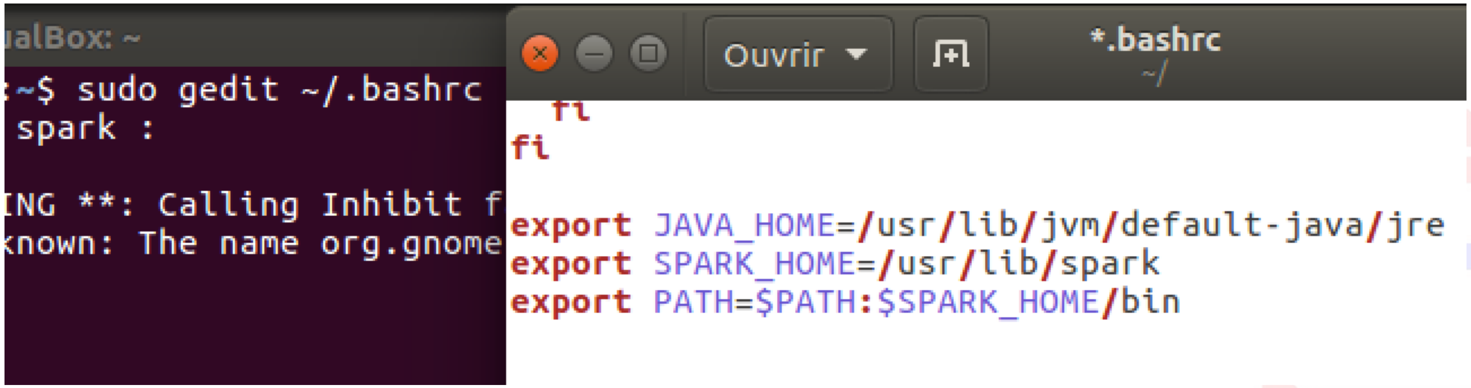
- Désarchiver le `spark-3.0.1-bin-hadoop2.7.tgz`

```
~$ tar xzvf ./Téléchargements/spark-3.0.1-bin-hadoop2.7.tgz
```

- Renommer le dossier `spark` et déplacer le sous `/usr/lib/`

```
~$ mv spark-3.0.1-bin-hadoop2.7/ spark
~$ sudo mv spark/ /usr/lib/
```

- Ajouter les variables d'environnement au `$PATH` à la fin du fichier `~/.bashrc` et relancer le terminal



LANCEMENT

Lancement de Spark

- Relancer le terminal pour la prise en compte des nouvelle variable et taper

```
~$ spark-shell
```

```
spark@spark-VirtualBox: ~
20/10/05 09:33:36 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
20/10/05 09:33:37 WARN NativeCodeLoader: Unable to load native-hadoop library fo
r your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLeve
l(newLevel).
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[*], app id = local-1601886831982
).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.0.1
      /_/

Using Scala version 2.12.10 (OpenJDK Server VM, Java 1.8.0_265)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

Lancement de Spark

Quitter l'ancien shell

```
scala> :quit
spark@spark-VirtualBox:~$ spark-shell --master "local[4]"
20/10/05 10:16:23 WARN Utils: Your hostname, spark-VirtualBox resolves to a loo
pback address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
20/10/05 10:16:23 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
 address
20/10/05 10:16:24 WARN NativeCodeLoader: Unable to load native-hadoop library f
or your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLev
el(newLevel).
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[4], app id = local-160188939962
7).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.0.1
      /_/

Using Scala version 2.12.10 (OpenJDK Server VM, Java 1.8.0_265)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

ur ce shell
naster sur le
threads

port du spark
eb UI

céder au
s un script
able sc

ssion avec la
variable spark

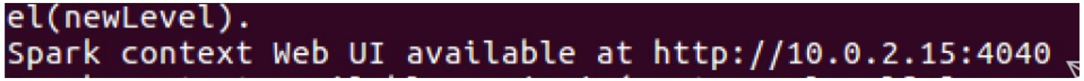
Lancement en 3 modes

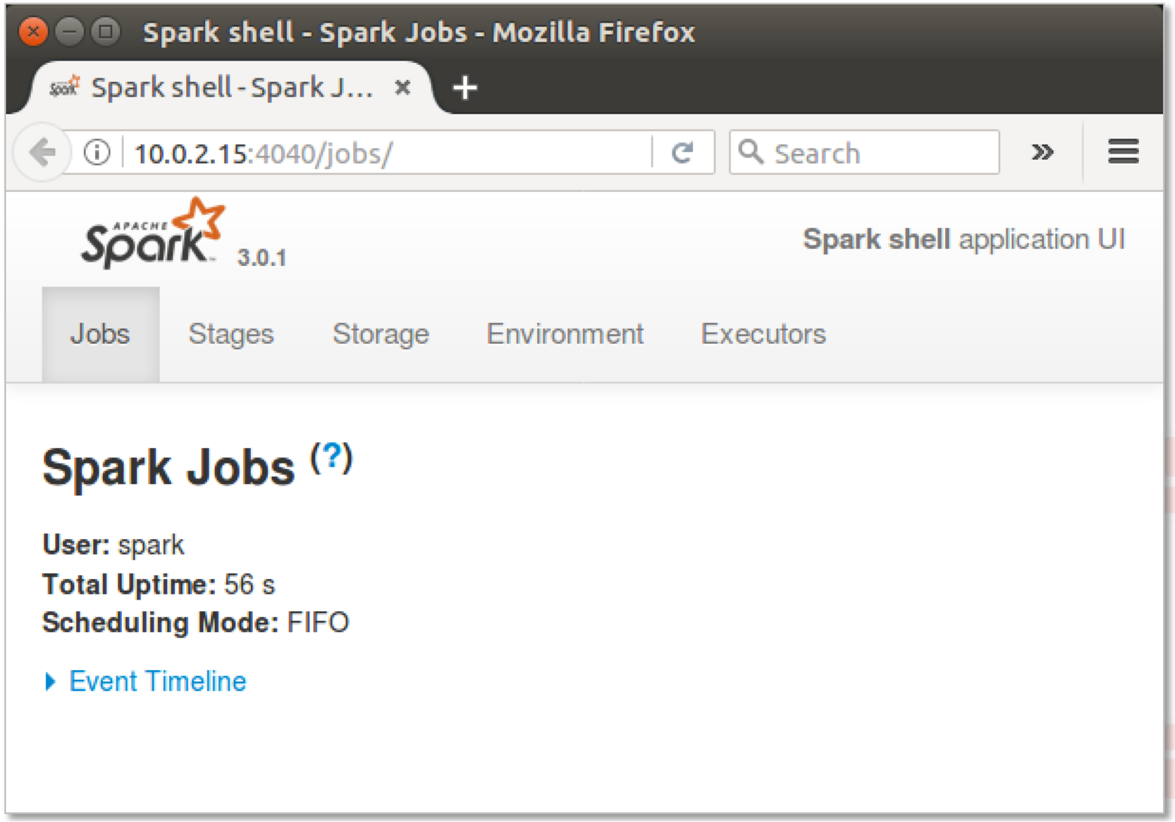
- L'option `--master` permet de préciser à quel type de `cluster manager` l'application Spark peut être envoyée.
- Spark peut fonctionner en se connectant à des `cluster managers` de types différents :
  - `--master spark://HOTE:PORT`: utilise le cluster manager autonome de Spark.
  - `--master mesos://HOTE:PORT`: se connecte à un cluster manager Mesos.
  - `--master yarn`: se connecte à un cluster manager Yarn.
  - `--master local[i]`: pas de cluster manager, Spark fonctionne en mode local. `i` spécifie le nombre d'executors (Worker) dans le cluster : local[1] ou local[4].

- Par défaut, si l'option `--master` n'est pas spécifiée, Spark fonctionne en mode local avec (nb d'executors = nb de cœurs logique de la machine).

Interface de contrôle Spark

- Spark dispose d'une interface Web qui permet de consulter les entrailles du système et de mieux comprendre ce qui est fait.
- Elle est accessible sur le port 4040





Versions de Spark : Scala, Python, Java

- pyspark permet de lancer Spark avec un shell python

```
spark@spark-VirtualBox:~$ spark-shell --master "local[4]
"
20/10/05 10:16:23 WARN Utils: Your hostname, spark-Virtu
alBox resolves to a loopback address: 127.0.1.1; using 1
0.0.2.15 instead (on interface enp0s3)
20/10/05 10:16:23 WARN Utils: Set SPARK_LOCAL_IP if you
need to bind to another address
20/10/05 10:16:24 WARN NativeCodeLoader: Unable to load
native-hadoop library for your platform... using builtin
-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/lo
g4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). Fo
r SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[4], app
id = local-1601889399627).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.0.1
      /_/

Using Scala version 2.12.10 (OpenJDK Server VM, Java 1.8
.0_265)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

```
spark@spark-VirtualBox:~$ pyspark
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
[GCC 5.4.0 20160609] on linux2
Type "help", "copyright", "credits" or "license" for more information.
20/10/05 13:41:48 WARN Utils: Your hostname, spark-VirtualBox resolves t
o a loopback address: 127.0.1.1; using 10.0.2.15 instead (on interface e
np0s3)
20/10/05 13:41:48 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to
another address
20/10/05 13:41:49 WARN NativeCodeLoader: Unable to load native-hadoop li
brary for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.pro
perties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use se
tLogLevel(newLevel).
20/10/05 13:41:52 WARN Utils: Service 'SparkUI' could not bind on port 4
040. Attempting port 4041.
/usr/lib/spark/python/pyspark/context.py:225: DeprecationWarning: Suppor
t for Python 2 and Python 3 prior to version 3.6 is deprecated as of Spa
rk 3.0. See also the plan for dropping Python 2 support at https://spark
.apache.org/news/plan-for-dropping-python-2-support.html.
  DeprecationWarning)
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.0.1
      /_/

Using Python version 2.7.12 (default, Nov 19 2016 06:48:10)
SparkSession available as 'spark'.
>>>
```

EXEMPLE : WORD COUNT

Exemple : wordcount avec Scala

- Soit le fichier `input.txt` dans le fichier où vous lancer le `spark-shell`
- Il faut exécuter les 3 commandes Scala

```
/** map */
var map = sc.textFile("input.txt").flatMap(line => line.split("
")).map(word => (word,1));

/** reduce */
var counts = map.reduceByKey(_ + _);

/** save the output to file */
counts.saveAsTextFile("output/")
```

On divise chaque ligne avec le séparateur « »

En utilisant la variable Spark context `sc` pour lire le fichier

On mappe à chaque mot un tuple `(word , 1)` 1 nombre d'occurrences

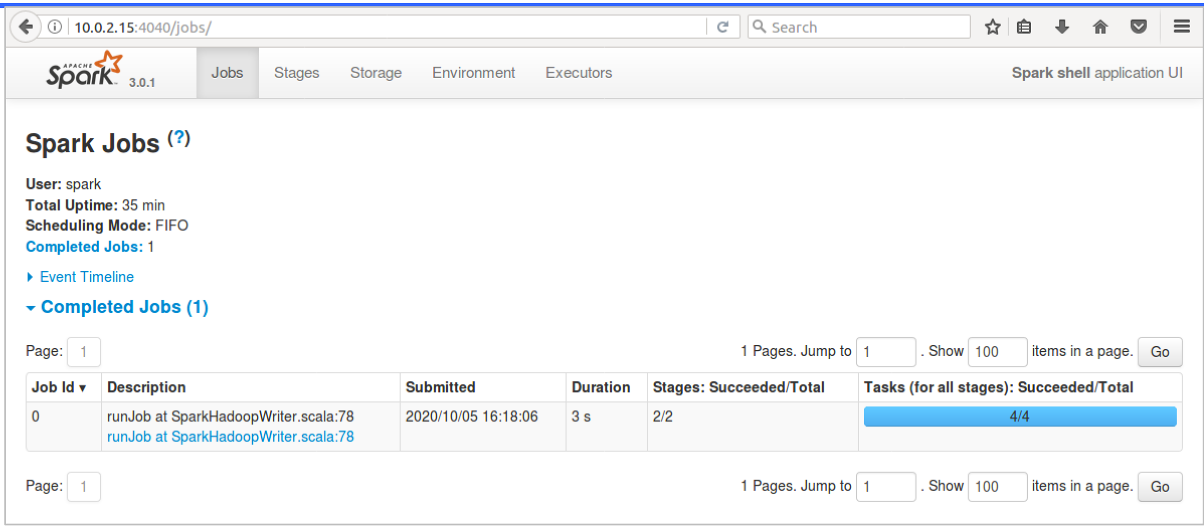
Sauvegarder le résultat dans des fichiers dans le répertoire `output`

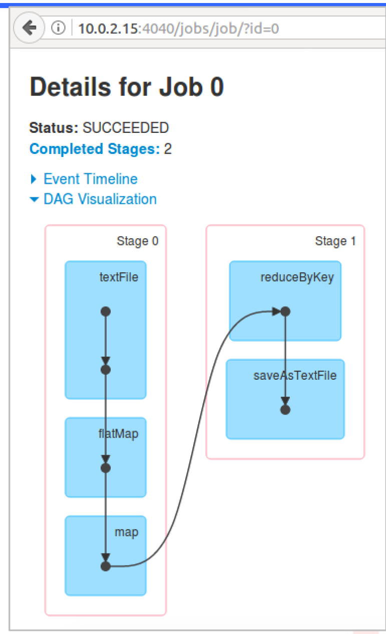
Exemple wordcount en Scala

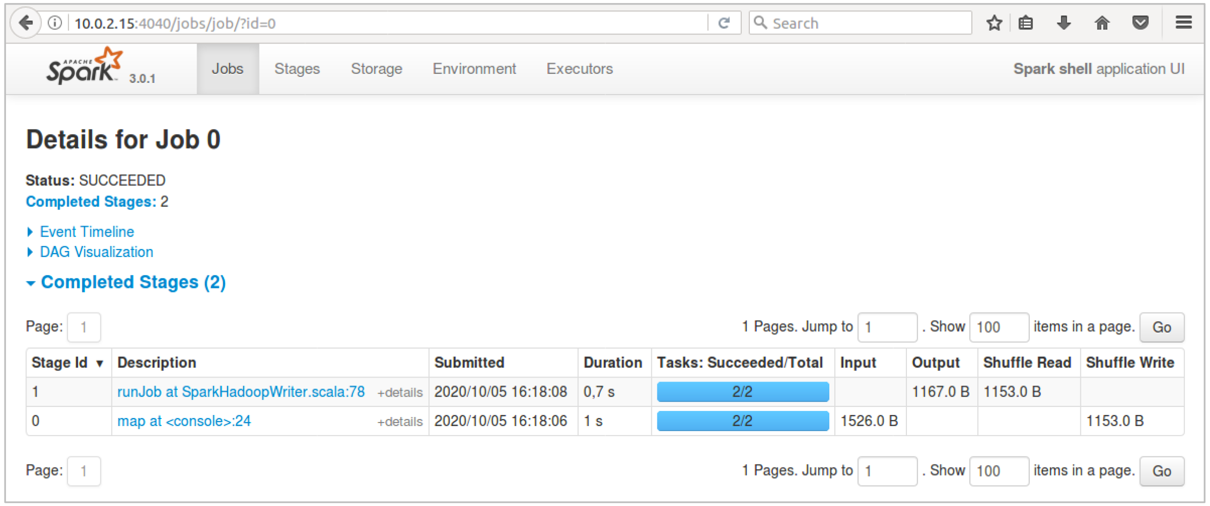
```
spark@spark-VirtualBox: ~/exemples/wordcount

      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.0.1
      /_/

Using Scala version 2.12.10 (OpenJDK Server VM, Java 1.8.0_265)
Type in expressions to have them evaluated.
Type :help for more information.

scala> var map = sc.textFile("input.txt").flatMap(line => line.split(" ")).map(w
ord => (word,1));
20/10/05 16:17:02 WARN SizeEstimator: Failed to check whether UseCompressedOops
is set; assuming yes
map: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[3] at map at <co
nsole>:24

scala> var counts = map.reduceByKey(_ + _);
counts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[4] at reduceByKey
at <console>:25

scala> counts.saveAsTextFile("output/")

scala>
```

```
spark@spark-VirtualBox: ~/exemples/wordcount
spark@spark-VirtualBox:~/exemples/wordcount$ ls output/
part-00000  part-00001  _SUCCESS
spark@spark-VirtualBox:~/exemples/wordcount$
```

Exemple : Spark Context Web UI

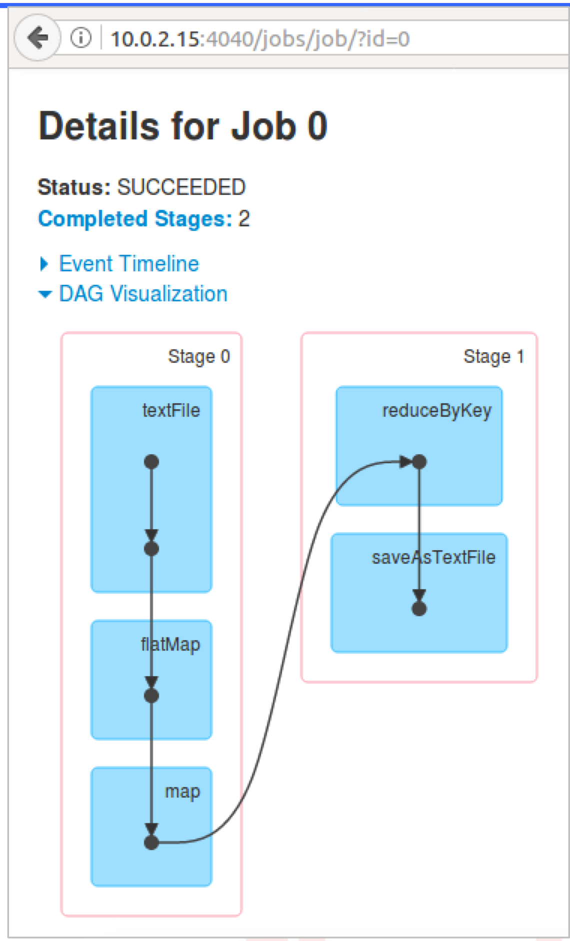




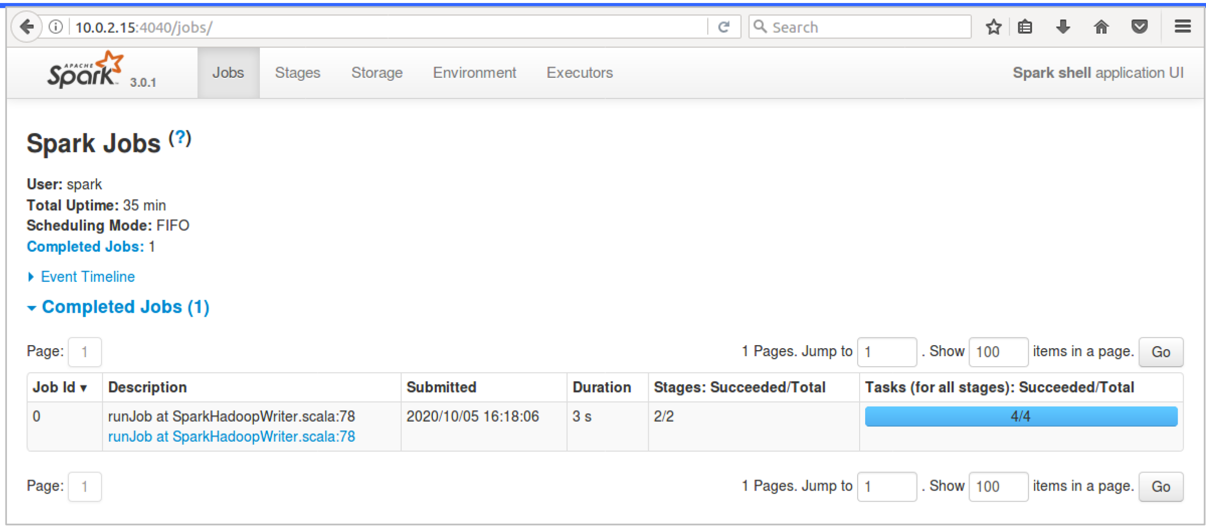


Spark Context Web UI - explication

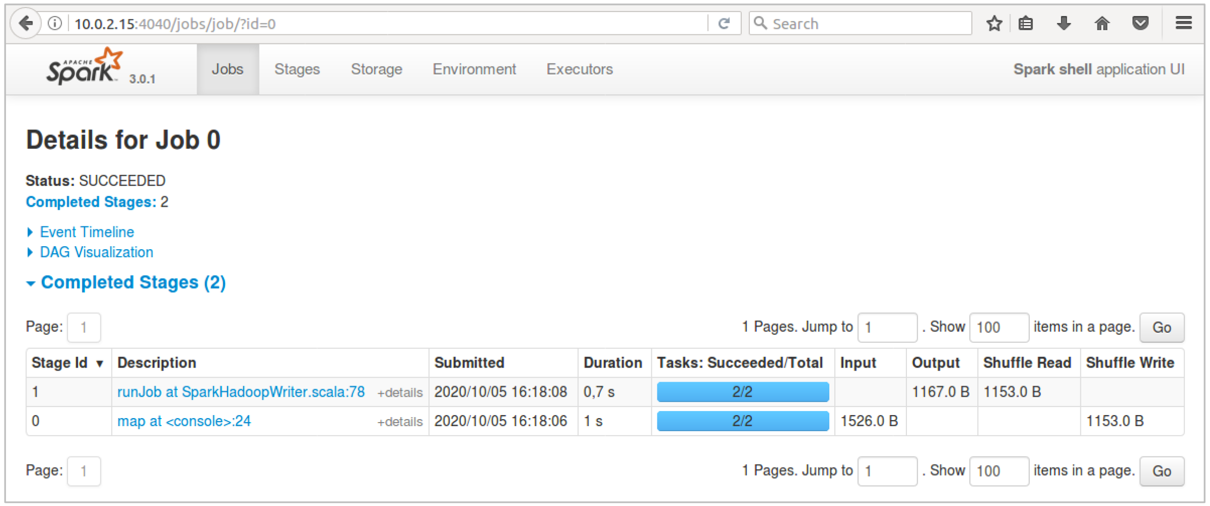
- Il y a 2 étapes (stages),
  - stage0, stage1
- une étape regroupe un ensemble d'opérations qu'il est possible d'exécuter sur une seule machine
- une étape est effectuée en **parallèle** sur les différents fragments des données
- Spark appelle tâche (task) l'exécution de l'étape sur un fragment particulier
- la séquence des étapes constitue un job.



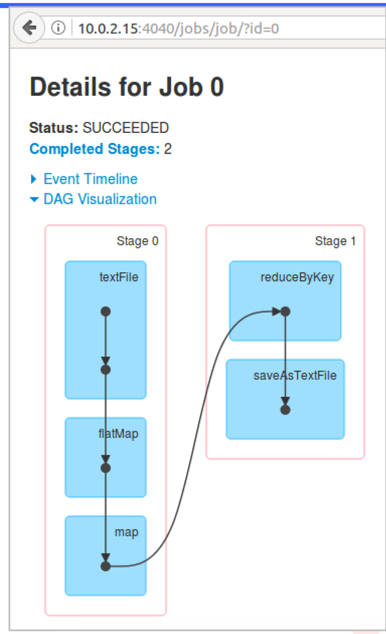
Exemple : Spark Context Web UI



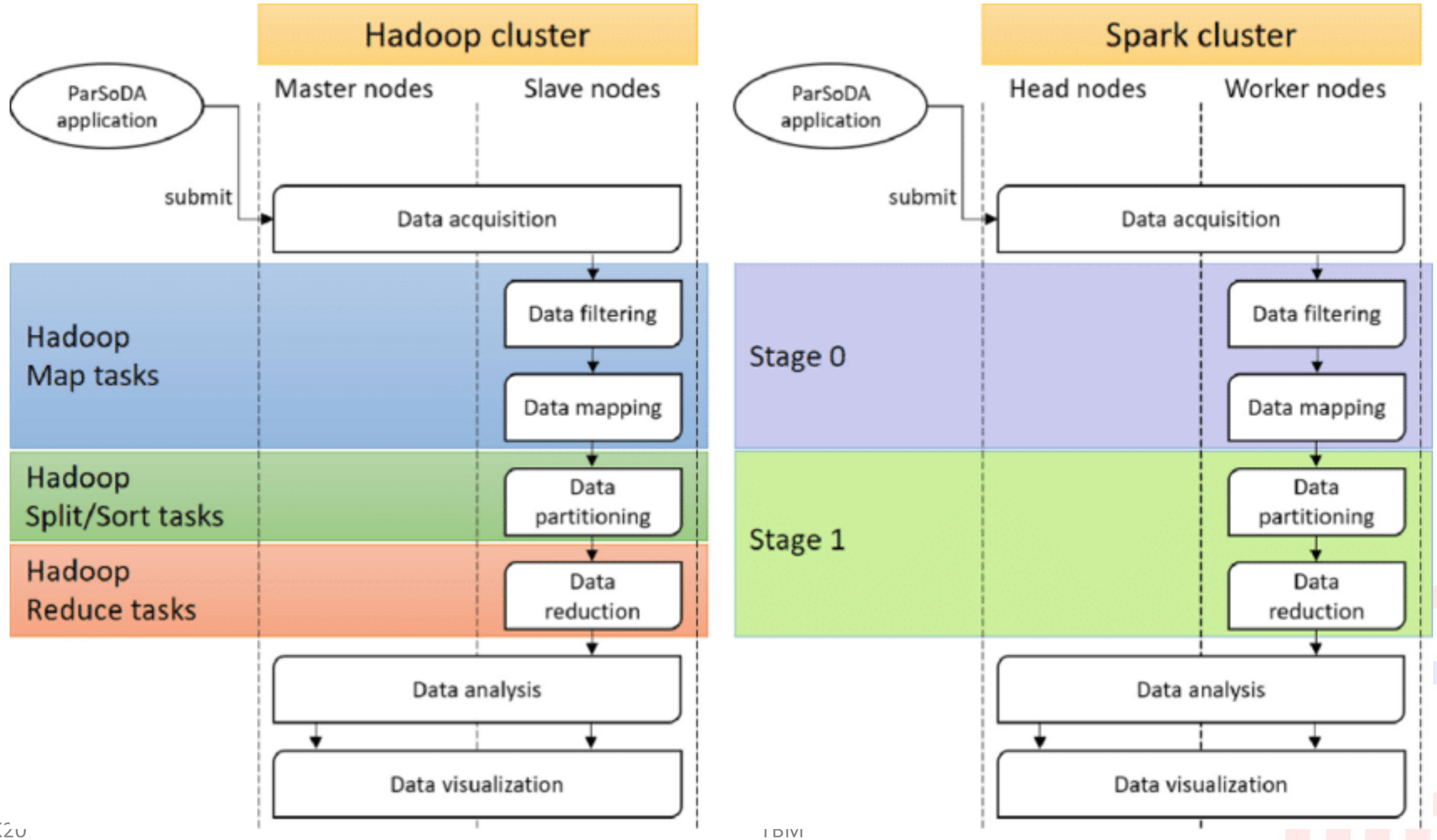
Job Id ▾	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2020/10/05 16:18:06	3 s	2/2	4/4



Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	runJob at SparkHadoopWriter.scala:78 +details	2020/10/05 16:18:08	0,7 s	2/2		1167.0 B	1153.0 B	
0	map at <console>:24 +details	2020/10/05 16:18:06	1 s	2/2	1526.0 B			1153.0 B



Spark vs. Hadoop : exécution



EXEMPLE : WORD COUNT - SHELL

Lancement de Spark python

```
spark@spark-VirtualBox: ~/exemples/wordcount
spark@spark-VirtualBox:~/exemples/wordcount$ pyspark --master local[4]
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
[GCC 5.4.0 20160609] on linux2
Type "help", "copyright", "credits" or "license" for more information.
20/10/05 17:31:54 WARN Utils: Your hostname, spark-VirtualBox resolves to a loop
back address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
20/10/05 17:31:54 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
20/10/05 17:31:57 WARN NativeCodeLoader: Unable to load native-hadoop library fo
r your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLeve
l(newLevel).
/usr/lib/spark/python/pyspark/context.py:225: DeprecationWarning: Support for Py
thon 2 and Python 3 prior to version 3.6 is deprecated as of Spark 3.0. See also
 the plan for dropping Python 2 support at https://spark.apache.org/news/plan-fo
r-dropping-python-2-support.html.
  DeprecationWarning)
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.0.1
      /_/

Using Python version 2.7.12 (default, Nov 19 2016 06:48:10)
SparkSession available as 'spark'.
>>>
```

Exemple : wordcount avec Python

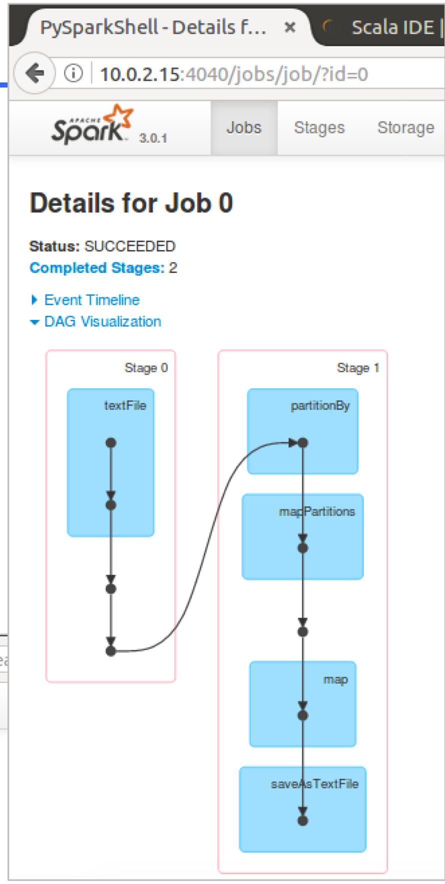
```python
input_file = sc.textFile("input.txt")
map = input_file.flatMap(lambda line: line.split(" ")).map(lambda
word: (word, 1))
counts = map.reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("output2/")
```

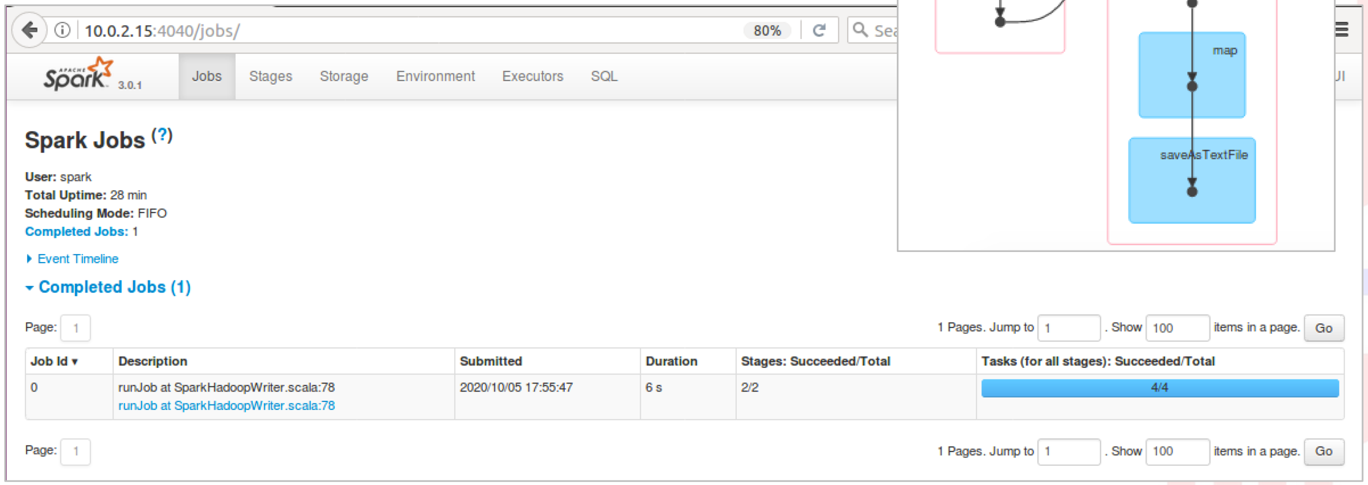
```
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.0.1
      /_/

Using Python version 2.7.12 (default, Nov 19 2016 06:48:10)
SparkSession available as 'spark'.
>>> input_file = sc.textFile("input.txt")
20/10/05 17:55:05 WARN SizeEstimator: Failed to check whether UseCompressedOops
is set; assuming yes
>>> map = input_file.flatMap(lambda line: line.split(" ")).map(lambda word: (wor
d, 1))
>>> counts = map.reduceByKey(lambda a, b: a + b)
>>> counts.saveAsTextFile("output2/")
>>>
```

```
spark@spark-VirtualBox:~/exemples/wordcount$ ls output2/
part-00000  part-00001  _SUCCESS
```

PySpark : visualisation

```
spark@spark-VirtualBox:~/exemples/wordcount$ cat output2/part-00000
(u'', 2)
(u'and', 2)
(u'followed', 1)
(u'is', 3)
(u'reduce', 1)
(u'fault-tolerant', 1)
(u'maintained', 1)
(u'map,', 1)
(u"Spark's", 1)
(u'technology', 1)
```

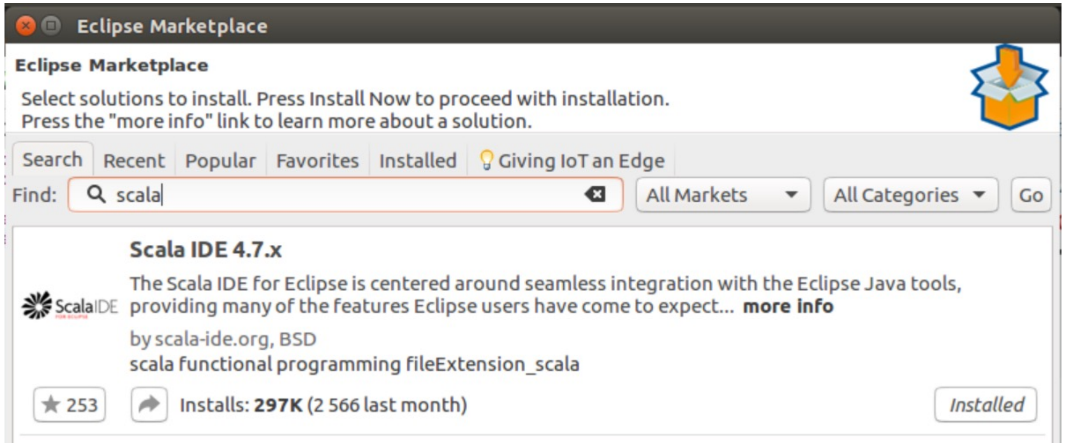




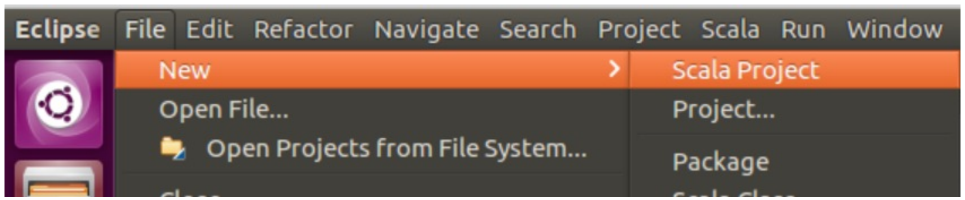
EXEMPLE : WORD COUNT AVEC APPLICATION

Exemple : wordcount avec application Scala

- Installer Scala IDE (4.7) sur Eclipse (4.7 Oxygen)

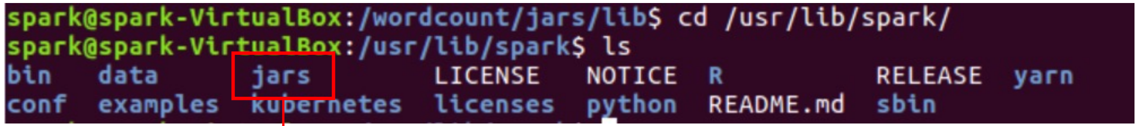


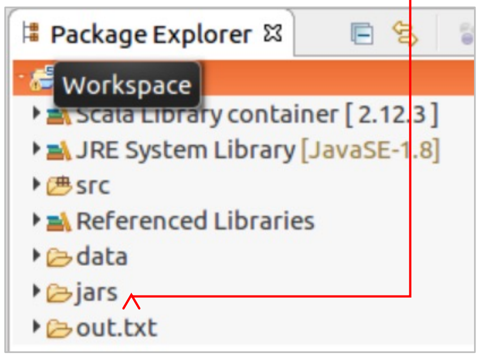
- Créer un projet Scala : `wordcount`

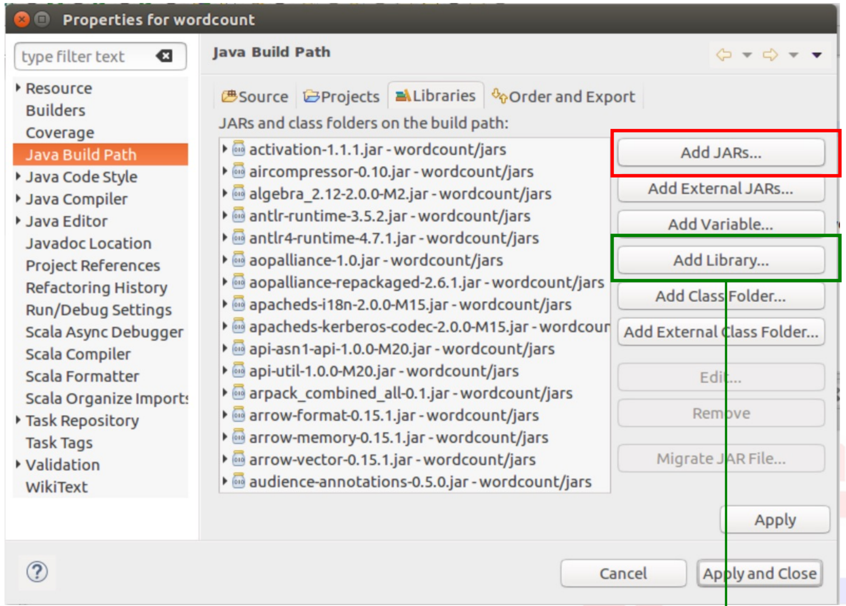


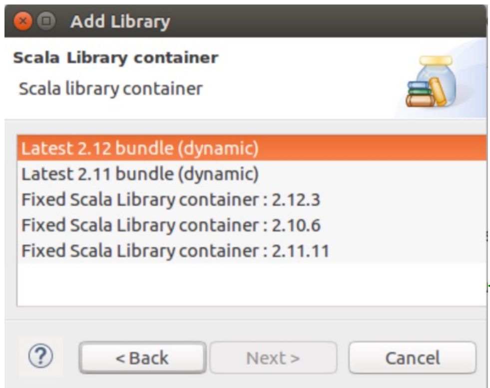
Exemple : wordcount avec application Scala

- Ajouter les Jars de Spark au projet







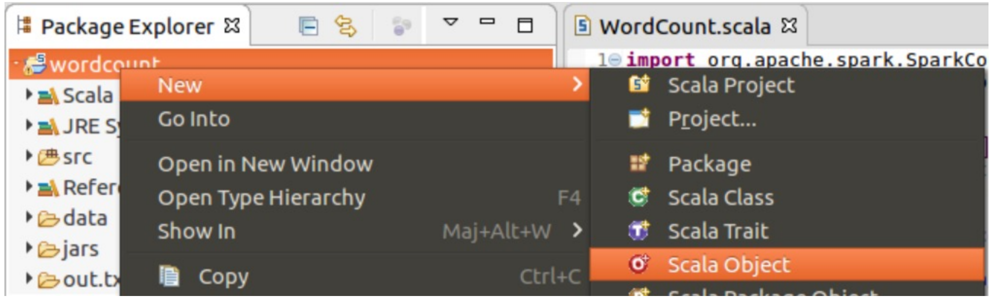


En cas d'erreur
*changer la version de bibliothèque de scala

- `$sudo mkdir /NOM_PROJET/REP_JAR`
- `$chmod 777 /NOM_PROJET/REP_JAR`

Exemple : wordcount avec application Scala

- Créer une nouvelle classe Scala

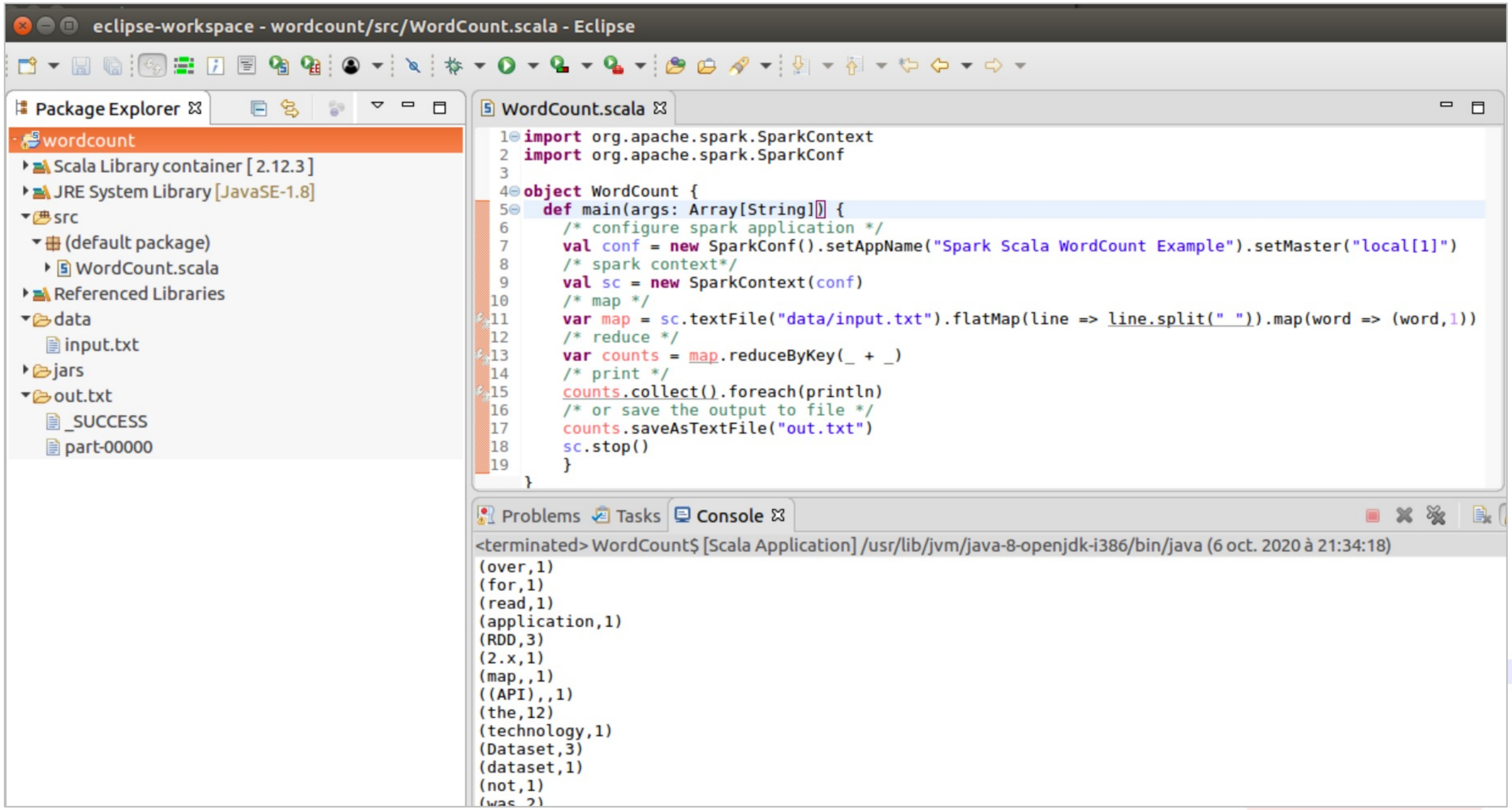


WordCount.scala

```scala
import org.apache.spark.SparkContext
import org.apache.spark.SparkConf

object WordCount {
  def main(args: Array[String]) {
    /* configure spark application */
    val conf = new SparkConf().setAppName("Spark Scala WordCount Example").setMaster("local[1]")
    /* spark context*/
    val sc = new SparkContext(conf)
    /* map */
    var map = sc.textFile("data/input.txt").flatMap(line => line.split(" ")).map(word => (word,1))
    /* reduce */
    var counts = map.reduceByKey(_ + _)
    /* print */
    counts.collect().foreach(println)
    /* or save the output to file */
    counts.saveAsTextFile("out.txt")
    sc.stop()
    }
}
```

Exemple : wordcount avec application Scala

- Exécuter la classe (Run As Scala Application)



EXEMPLE : WORD COUNT AVEC APPLICATION

Exemple : wordcount – Application python

- Ecrire un programme python

```python
import sys

from pyspark import SparkContext, SparkConf

if __name__ == "__main__":
        # create Spark context with Spark configuration
        conf = SparkConf().setAppName("Word Count - Python").set
("spark.hadoop.yarn.resourcemanager.address", "10.0.2.15:8032")
        sc = SparkContext(conf=conf)

        # read in text file and split each document into words
        words = sc.textFile("/home/spark/exemples/wordcount/input.txt").flatMap(lambda line:
line.split(" "))

        # count the occurrence of each word
        wordCounts = words.map(lambda word: (word, 1)).reduceByKey(lambda a,b:a +b)

        wordCounts.saveAsTextFile("/home/spark/exemples/wordcount/python-app/output/")
```

- Exécuter la commande

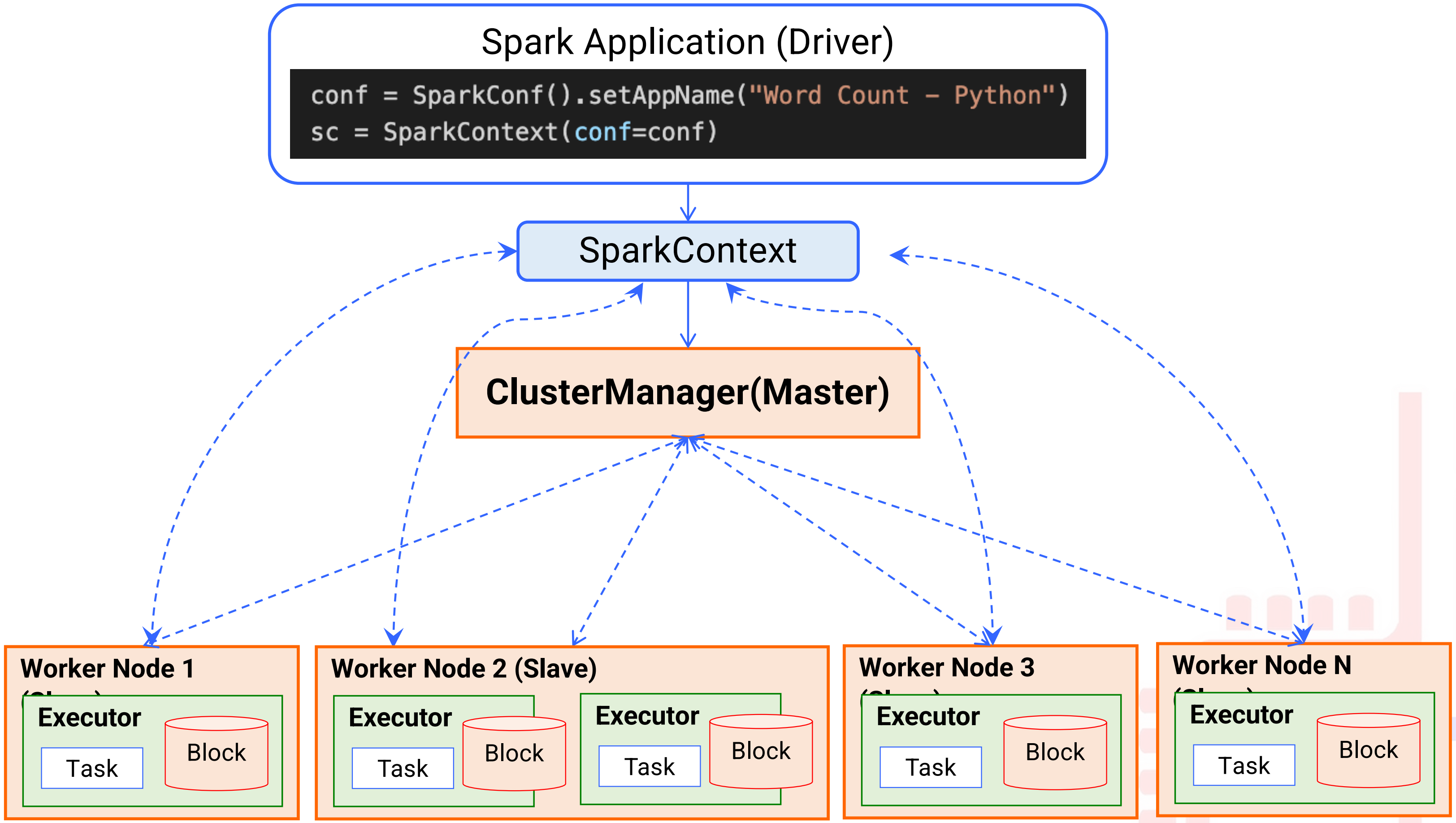
```
$ spark-submit wordcount.py
```

- Résultat

```
spark@spark-VirtualBox:~/exemples/wordcount/python-app$ ls -l output/
total 4
-rw-r--r-- 1 spark spark 1571 أكت 6 17:39 part-00000
-rw-r--r-- 1 spark spark    0 أكت 6 17:39 _SUCCESS
```

CONFIGURATION DE SPARK

Batch Processing



Configurer un Spark Cluster

- **Configurer Spark Master Node**
  - Aller au répertoire de configuration de spark SPARK_HOME/conf/
  - Editer le fichier spark-env.sh (à Créer s'il n'existe pas en copiant spark-env.sh.template)

```
spark@spark-VirtualBox: /usr/lib/spark/conf
spark@spark-VirtualBox:~$ cd /usr/lib/spark/conf
spark@spark-VirtualBox:/usr/lib/spark/conf$ ls
fairscheduler.xml.template   slaves.template
log4j.properties.template    spark-defaults.conf.template
metrics.properties.template  spark-env.sh.template
spark@spark-VirtualBox:/usr/lib/spark/conf$ cp spark-env.sh.template spark-env.sh
```

  - Ajouter la variable SPARK_MASTER_HOST

```
spark-env.sh (/usr/lib/spark/conf) - gedit
Ouvrir
# Options for the daemons used in the standalone deploy mode
# - SPARK_MASTER_HOST, to bind the master to a different IP address or hostname
export SPARK_MASTER_HOST='127.0.0.1'
# - SPARK_MASTER_PORT / SPARK_MASTER_WEBUI_PORT, to use non-default ports for the master
```

Configurer un Spark Cluster

- Lancer spark comme Master en lançant la commande
  `SPARK_HOME/sbin/start-master.sh`

```
spark@spark-VirtualBox: /usr/lib/spark/sbin
spark@spark-VirtualBox:/usr/lib/spark$ cd sbin
spark@spark-VirtualBox:/usr/lib/spark/sbin$ ./start-master.sh
starting org.apache.spark.deploy.master.Master, logging to /usr/lib/spark/logs/spark-s
park-org.apache.spark.deploy.master.Master-1-spark-VirtualBox.out
spark@spark-VirtualBox:/usr/lib/spark/sbin$
```

  - Output du fichier log `.out`

```
Spark Command: /usr/lib/jvm/default-java/jre/bin/java -cp /usr/lib/spark/conf/:/usr/lib/spark/jars/
* -Xmx1g org.apache.spark.deploy.master.Master --host 127.0.0.1 --port 7077 --webui-port 8080
========================================
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
20/10/11 13:38:19 INFO Master: Started daemon with process name: 4289@spark-VirtualBox
20/10/11 13:38:19 INFO SignalUtils: Registered signal handler for TERM
20/10/11 13:38:19 INFO SignalUtils: Registered signal handler for HUP
```

- **Lancer Spark Slave (Worker) Node**
  - Lancer un Worker : `$ ./start-slave.sh spark://<master.ip.address>:7077`

```
spark@spark-VirtualBox:/usr/lib/spark/sbin$ ./start-slave.sh spark://127.0.0.1:7077
starting org.apache.spark.deploy.worker.Worker, logging to /usr/lib/spark/logs/spark-spark-org.apac
he.spark.deploy.worker.Worker-1-spark-VirtualBox.out
```

Configurer un Spark Cluster

- Output du log Worker

```
spark@spark-VirtualBox: /usr/lib/spark/sbin
Spark Command: /usr/lib/jvm/default-java/jre/bin/java -cp /usr/lib/spark/conf/:/usr/lib/spark/jars/
* -Xmx1g org.apache.spark.deploy.worker.Worker --webui-port 8081 spark://127.0.0.1:7077
========================================
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
20/10/11 13:41:38 INFO Worker: Started daemon with process name: 4366@spark-VirtualBox
20/10/11 13:41:38 INFO SignalUtils: Registered signal handler for TERM
20/10/11 13:41:38 INFO SignalUtils: Registered signal handler for HUP
20/10/11 13:41:38 INFO SignalUtils: Registered signal handler for INT
20/10/11 13:41:38 WARN Utils: Your hostname, spark-VirtualBox resolves to a loopback address: 127.0
.1.1; using 10.0.2.15 instead (on interface enp0s3)
20/10/11 13:41:38 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
20/10/11 13:41:40 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...
using builtin-java classes where applicable
20/10/11 13:41:40 INFO SecurityManager: Changing view acls to: spark
20/10/11 13:41:40 INFO SecurityManager: Changing modify acls to: spark
20/10/11 13:41:40 INFO SecurityManager: Changing view acls groups to:
20/10/11 13:41:40 INFO SecurityManager: Changing modify acls groups to:
20/10/11 13:41:40 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled;
 users  with view permissions: Set(spark); groups with view permissions: Set(); users  with modify
permissions: Set(spark); groups with modify permissions: Set()
20/10/11 13:41:41 INFO Utils: Successfully started service 'sparkWorker' on port 45217.
20/10/11 13:41:41 INFO Worker: Starting Spark worker 10.0.2.15:45217 with 1 cores, 3.5 GiB RAM
20/10/11 13:41:41 INFO Worker: Running Spark version 3.0.1
20/10/11 13:41:41 INFO Worker: Spark home: /usr/lib/spark
20/10/11 13:41:41 INFO ResourceUtils: ==============================================================
=
20/10/11 13:41:41 INFO ResourceUtils: Resources for spark.worker:

20/10/11 13:41:41 INFO ResourceUtils: ==============================================================
=
20/10/11 13:41:42 INFO Utils: Successfully started service 'WorkerUI' on port 8081.
20/10/11 13:41:42 INFO WorkerWebUI: Bound WorkerWebUI to 0.0.0.0, and started at http://10.0.2.15:8
081
20/10/11 13:41:42 INFO Worker: Connecting to master 127.0.0.1:7077...
```

Configurer un Spark Cluster

- **Configurer Spark Slave (Worker) Node**
  - Aller au répertoire de configuration de spark `SPARK_HOME/conf/`
  - Editer le fichier `slaves` (à Créer s'il n'existe pas en copiant `slaves.template`)

```
# A Spark Worker will be started on each of the machines listed below.
localhost
localhost
```